

Briefing für Claude Code — Simulationsumgebung für marktbasierte Aufgabenzuweisung

Market-Based Task Allocation for Multi-Robot Rescue Exploration ·
Praktikum Verteilte Robotiksysteme, TU Darmstadt

Halil Elbay, Paul Butter

11. August 2026

Inhaltsverzeichnis

Wie dieses Dokument zu benutzen ist	2
1. Fachlicher Hintergrund	3
1.1 Das Problem	3
1.2 Die Plattform	3
1.3 Was gebaut werden soll — und was ausdrücklich nicht	3
2. Search-and-Rescue-Kontext: warum jede Komponente existiert	4
2.1 Die Overhead-Kamera vertritt das GPS	4
2.2 Die Bid-Parameter sind Ressourcenmanagement	4
2.3 Was die einzelnen Modellelemente in der Realität bedeuten	5
2.4 Ökologische und soziale Effekte	6
3. Architektur	7
3.1 Leitprinzip	7
3.2 Die harte Regel	7
3.3 Die Backend-Schnittstelle	7
4. Die Kostenfunktion — korrigiert	9
4.1 Fehler 1: inkonsistente Vorzeichen	9
4.2 Fehler 2: δ ist nicht identifizierbar	9
4.3 Endgültige Formulierung	9
4.4 Wichtig: p ist keine Konstante	10
5. Prioritätsdynamik	11
5.1 Die gemeinsame Formel	11
5.2 Warum diese Form und keine andere	11
5.3 Folge für das Protokoll — nicht optional	11
5.4 SURVEY als vierter Task-Typ	12
5.5 Basis-Prioritäten je Task-Typ	12
5.6 Aging gegen Aushungerung	12
5.7 Preemptionskaskaden	13
5.8 Ereignisinjektion — die interessanteste Messung	13
5.9 Was das für den Bericht bedeutet	13
5.10 Einschränkung für das Labor	13

6. Arbeitspaket 1 — allocation/	14
6.1 tasks.py	14
6.2 bidding.py	14
6.3 messages.py	14
6.4 auction.py	15
6.5 strategies.py	16
7. Arbeitspaket 2 — sim/ und backends/abstract.py	17
7.1 Bewegungsmodell	17
7.2 Batteriemodell	17
7.3 Detektionsmodell	17
7.4 Kommunikationsmodell	17
7.5 Zwei Raster: Coverage und SURVEY	17
7.6 Ereignisinjektion	18
7.7 Explorationsverhalten	18
7.8 Zeitschleife	18
8. Arbeitspaket 3 — metrics/	19
8.1 Zwei CSV-Dateien pro Lauf	19
8.2 Die vier Metriken	19
8.3 Metriken der persistenten Überwachung	19
9. Invarianten und Tests	20
10. Vorgaben für den Quellcode	21
11. Reihenfolge der Arbeitspakete	22
12. Glossar	23

Wie dieses Dokument zu benutzen ist

Dieses Dokument ist die vollständige Arbeitsanweisung für die Entwicklung der Python-Simulationsumgebung mit Claude Code. Es enthält den fachlichen Hintergrund, die Bedeutung jeder Komponente im Kontext einer realen Search-and-Rescue-Operation (SAR), die Architekturvorgaben und die Spezifikation jedes Moduls.

Fassung 2 (11. August 2026): ergänzt um die Prioritätsdynamik — Abschnitt 5 ist neu, die Abschnitte 4.4, 6.1, 6.3, 6.4, 7.5–7.8, 8.3, 9 und 11 wurden angepasst.

Empfohlene Nutzung: Dieses Dokument als BRIEFING.md in das Repository legen und Claude Code zu Beginn jeder Sitzung darauf verweisen. Die Abschnitte 5 bis 11 sind so geschrieben, dass sie einzeln als Arbeitspaket übergeben werden können.

Wichtiger Hinweis zur Arbeitsweise: Claude Code macht das *Schreiben* von Code billig, nicht das *Verstehen*. Der Quellcode wird im Praktikumsbericht bewertet — explizit auf Struktur, Effizienz und darauf, ob alle Zeilen notwendig sind. Außerdem muss jede Designentscheidung im Bericht verteidigt werden können. Deshalb gilt: nach jedem Arbeitspaket den erzeugten Code lesen, verstehen und **kürzen**. Generierter Code neigt zu Redundanz und zu defensiven Konstruktionen, die Fehler verdecken statt sie sichtbar zu machen.

1. Fachlicher Hintergrund

1.1 Das Problem

Ein Team autonomer Roboter erkundet gemeinsam ein Areal. Dabei werden **Regions of Interest** (RoIs) entdeckt: Objekte, die bewegt werden müssen, Gefahrenstellen, Fundorte. Jede RoI erzeugt eine Aufgabe. Nun muss entschieden werden, *welcher Roboter welche* Aufgabe übernimmt.

Zwei naheliegende Antworten sind beide schlecht:

- **Immer der nächstgelegene Roboter** ignoriert, dass dieser Roboter vielleicht schon drei Aufgaben offen hat oder dass sein Akku fast leer ist. Er wird systematisch überlastet und fällt als Erster aus.
- **Zufällige Zuweisung** verschwendet Fahrzeit und damit die knappste Ressource überhaupt.

Marktbasierte Aufgabenzuweisung ist der Mittelweg: Der entdeckende Roboter schreibt die Aufgabe aus, alle Roboter berechnen aus ihrem *eigenen* lokalen Zustand ein Gebot, und die besten Gebote gewinnen. Es gibt keine zentrale Instanz, die entscheidet.

1.2 Die Plattform

Pi-Puck = e-puck2 (differentiell angetriebener Kleinroboter, ca. 7 cm Durchmesser, IR-Distanzsensoren, Kamera) plus aufgesetzter Raspberry Pi für Rechenleistung und WLAN. Die Roboter kommunizieren untereinander über UDP.

1.3 Was gebaut werden soll — und was ausdrücklich nicht

Es entstehen drei Ausführungsebenen mit **identischer Auktionslogik**:

Ebene	Werkzeug	Zweck	Status
Tier 0	diese Python-Umgebung	tausende Läufe, Parametersuche	wird gebaut
Tier 1	Webots (E-puck2 + Pi-puck PROTO)	Validierung mit echter Dynamik	existiert bereits
Tier 2	echte Pi-Pucks	Realitätsnachweis	Hardware

Tier 0 ist bewusst unrealistisch. Die Roboter sind Punkte. Es gibt keine Kollisionsphysik, kein Radschlupfmodell, keine Raycasts. Der einzige Zweck dieser Ebene ist Geschwindigkeit: Wenn ein Lauf 1,5 Sekunden dauert statt 3 Minuten, sind 8.000 Läufe eine Kaffeepause statt eines Monats.

Nicht bauen (das erledigt Webots besser und kostet hier nur Zeit ohne Gegenwert):

- Physik-Engine, Kollisionsauflösung, Kontaktkräfte
- Kinematisches Differentialantriebsmodell mit Radgeschwindigkeiten
- IR-Sensor-Raycasting gegen Geometrie
- Kamerabildsimulation
- ROS, asyncio-Framework, GUI, Web-Dashboard, Machine Learning

2. Search-and-Rescue-Kontext: warum jede Komponente existiert

Dieser Abschnitt ist nicht Beiwerk. Er liefert die Begründungen, die im Bericht in der Introduction, der Problem Formulation und der Discussion gebraucht werden. Jede Designentscheidung im Code hat eine Entsprechung in der realen Anwendung.

2.1 Die Overhead-Kamera vertritt das GPS

Im Labor liefert eine Deckenkamera die Positionen aller Roboter. Das wirkt wie Schummeln — ist es aber nicht, sondern die naheliegende Emulation dessen, was in einer realen Operation die Lokalisierungsinfrastruktur leistet:

- **Im Freien:** GNSS (GPS/Galileo), gegebenenfalls mit RTK-Korrektur
- **Im Trümmerfeld ohne Satellitenempfang:** ein Aufklärungs-UAV, das die Einsatzfläche überfliegt und Positionen relativ zu einer vorab erstellten Karte meldet; oder aufgestellte UWB-Anker

Der entscheidende konzeptionelle Punkt, der im Bericht klar herausgearbeitet werden muss:

Lokalisierung darf zentralisiert sein. Entscheidungsfindung darf es nicht.

Ein GPS-Ausfall degradiert die Navigationsgenauigkeit — alle Roboter arbeiten weiter, nur ungenauer. Der Ausfall einer *Kommandozentrale*, die Aufgaben zuweist, legt dagegen die gesamte Flotte still. In einer Katastrophenlage ist genau das der wahrscheinliche Fall: Funkabschattung durch Beton, Mehrwegeausbreitung, überlastete Netze, ausgefallene Infrastruktur. Deshalb ist die Auktion dezentral, obwohl die Lokalisierung es nicht ist.

Konsequenz für den Code: Das Simulationsmodell darf Roboterpositionen als bekannt annehmen (Ground Truth mit optionalem Rauschen). Es darf **niemals** eine globale Instanz enthalten, die Aufgaben zuweist. Jede Zuweisung entsteht ausschließlich aus dem Nachrichtenaustausch zwischen Robotern.

2.2 Die Bid-Parameter sind Ressourcenmanagement

Die Kostenfunktion enthält vier Terme. Jeder entspricht einer realen Einsatzüberlegung.

Distanz (α) — die goldene Stunde. Die Überlebenswahrscheinlichkeit Verschütteter fällt monoton mit der Zeit, in den ersten 24 Stunden besonders steil. Anfahrtszeit ist damit unmittelbar in Rettungsertrag umrechenbar. Deshalb ist der Distanzterm der Referenzterm und wird auf $\alpha = 1$ normiert; alle anderen Gewichte werden relativ dazu ausgedrückt.

Auslastung (β) — Schichtrotation. Wenn stets die schnellste Einheit alles übernimmt, verschleißt sie zuerst und fällt zuerst aus. Bei menschlichen Einsatzkräften ist Rotation selbstverständlich; bei Robotern muss sie in die Zuweisungsregel hineinkonstruiert werden. Der Lastterm ist der Preis, den man in Fahrzeit zahlt, um Ausfälle zu vermeiden.

Batterie (γ) — die Rückkehrreserve. Ein Roboter, der mitten im Einsatz liegen bleibt, ist nicht nur ein Verlust an Kapazität. Er wird zum Hindernis und im Extremfall selbst zum Bergungsobjekt. Die Luftrettung kennt dafür nicht verhandelbare Reservekraftstoffregeln. Der Batterieterm ist die Übersetzung dieses Prinzips in eine Gebotsfunktion.

Priorität (δ) — Triage. Nicht alle Fundstellen sind gleich dringlich. Wird während eines laufenden Auftrags etwas Dringlicheres gefunden, muss Umdisponieren möglich sein. Genau das ist medizinische Triage. Die Hysterese im Preemptionskriterium hat ebenfalls eine reale Entsprechung: ständiges Umdisponieren kostet mehr Fahrzeit, als der Prioritätsgewinn einbringt — auch reale Einsatzleitungen ändern eine Zuweisung nicht wegen eines marginalen Unterschieds.

Warum die Parameter überhaupt gesucht werden müssen. Die vier Ziele stehen im Konflikt. Man kann nicht gleichzeitig die Bearbeitungszeit minimieren, die Abnutzung gleichmäßig verteilen und die Flottenlaufzeit maximieren. Die Parameter sind der Regler, über den eine Einsatzleitung diese Abwägung **vor** dem Einsatz einstellt.

Daraus folgt unmittelbar, was gesucht wird: **nicht ein optimaler Parametersatz, sondern eine Pareto-Front.** Welcher Punkt auf dieser Front richtig ist, hängt vom Szenario ab — kurze intensive Operation gegen mehrtägigen Dauereinsatz. Und daraus folgt der ganze Zweck der Simulation: In einer echten Katastrophe kann man keine Parameter durchprobieren.

2.3 Was die einzelnen Modellelemente in der Realität bedeuten

Element im Modell	Reale Entsprechung
Dezentrale Auktion	Kein Single Point of Failure; funktioniert in Teilgruppen weiter, wenn die Funkverbindung zerfällt
UDP ohne Zustellgarantie	Reale Funkverbindungen im Trümmerfeld: Paketverlust, Abschattung, keine Quittung
$n_{req} = 2-3$ (Push)	Ein Trümmerteil, das eine einzelne Einheit nicht bewegen kann — Kräfteansatz
$n_{req} = 3$ (Surround)	Absperren und Eingrenzen einer Gefahrenstelle (Gasaustritt, Brandherd)
$n_{req} = 1$ (Guard)	Fundstelle markieren und beobachten, bis menschliche Kräfte eintreffen
Kommunikations-Overhead als Metrik	Funkspektrum ist im Katastrophenfall knapp und wird mit Menschen geteilt. Ein Protokoll mit hunderten Nachrichten pro Aufgabe skaliert nicht auf 50 Einheiten
Coverage-Rate als Metrik	Durchsuchte Fläche pro Zeit ist direkt proportional zur Wahrscheinlichkeit, Überlebende rechtzeitig zu finden
Lastungleichheit als Metrik	Prognose der Ausfallreihenfolge: die am stärksten belastete Einheit fällt zuerst aus
Detektionsradius statt Kamerabild	Reichweite eines realen Sensors (Wärmebild, CO ₂ -Detektor, Kamera)
SURVEY-Task auf veralteten Zellen	Ein Bereich, der lange nicht eingesehen wurde, ist informationsarm — dort kann sich seit dem letzten Blick etwas verändert haben (Nachbeben, Brandausbreitung, ein Überlebender hat sich bewegt)
Wachsende Priorität mit der Wartezeit	Erwartete Ereignisdichte seit der letzten Beobachtung; im Einsatz die Grundlage dafür, Suchgebiete erneut abzulaufen statt sie als „erledigt“ abzuhaken
Basis-Priorität je Task-Typ	Einsatztaktische Rangfolge: eine sich ausbreitende Gefahr, eine lokalisierte Fundstelle und eine blockierte Zugangsroute haben nicht denselben Wert

Element im Modell	Reale Entsprechung
Aging (Priorität steigt auch nach Ankündigung)	Verhindert, dass niedrig eingestufte Aufgaben bei ausgelasteter Flotte dauerhaft liegen bleiben
Bewusster Reality Gap	Ein in Simulation optimaler Parametersatz kann real versagen — Radschlupf, Funklatenz, Sensorrauschen. Deshalb wird über drei Ebenen validiert

2.4 Ökologische und soziale Effekte

Für diesen Aspekt sind im Bewertungsschema eigene 5 Punkte vorgesehen. Vier Ansatzpunkte, die sich konkret auf die eigene Technologie beziehen lassen:

Algorithmische Priorisierung von Menschen. Der Parameter p entscheidet implizit, wem zuerst geholfen wird. Wer legt p fest, nach welchen Kriterien, und wer haftet für eine Fehlgewichtung? Ein Verfahren, das Prioritäten *automatisch* aus Sensordaten ableitet, verschiebt eine ethische Entscheidung in eine Software. Die vertretbare Position ist eine explizite Trennung: die Maschine schlägt eine Reihenfolge vor, ein Mensch verantwortet sie.

Ressourcen- und Energiebilanz. Eine Roboterflotte verbraucht Lithium, Seltene Erden und Fertigungsenergie und wird nach wenigen Jahren zu Elektroschrott. Die ehrliche Abwägung lautet: Der Aufwand rechtfertigt sich nur, wenn die Flotte tatsächlich Leben rettet oder Menschen aus tödlichen Umgebungen heraushält. Der Batterieterm in der Kostenfunktion ist übrigens auch ein Effizienzterm — Lastverteilung verlängert Gerätelebensdauer und reduziert Ersatzbeschaffung.

Dual-Use der Infrastruktur. Ein Overhead-Kamerasystem, das Roboter lokalisiert, lokalisiert ebenso gut Menschen. Die für SAR entwickelte Aufklärungs- und Koordinationsinfrastruktur ist ohne technische Änderung für Überwachung verwendbar. Das ist keine hypothetische Sorge, sondern der Regelfall bei Sensorik im öffentlichen Raum.

Verhältnis zu menschlichen Einsatzkräften. Der starke Fall für Rettungsrobotik ist nicht Kostenersparnis, sondern der Einsatz dort, wo Menschen nicht hin können oder sollen — Strahlung (Fukushima), Einsturzgefahr, Giftgas. Ein Argument, das mit Personaleinsparung wirbt, ist sowohl schwächer als auch anfälliger für Kritik.

3. Architektur

3.1 Leitprinzip

Die Auktionslogik wird **einmal** geschrieben und läuft unverändert in allen drei Ebenen. Getauscht wird nur das Backend darunter.

```
allocation/          # plattformunabhängig, NULL Abhängigkeiten nach unten
  __init__.py
  tasks.py           # Task-Definitionen, Typen, Zustände
  bidding.py         # Kostenfunktion und Normierungen
  auction.py         # Auktionsprotokoll als endlicher Automat
  strategies.py      # market | greedy | random
  messages.py        # Nachrichtenformate, Serialisierung

backends/
  base.py            # abstrakte Schnittstelle (Protocol)
  abstract.py        # Tier 0: Punktroboter, schnell
  webots.py          # Tier 1 (Phase 2, hier noch nicht)
  pipuck.py          # Tier 2 (Phase 5, hier noch nicht)

sim/
  world.py           # Arena, RoI-Verteilung, Zeitschleife
  robot.py           # Punktroboter-Zustand und Bewegung
  comms.py           # Nachrichtenkanal mit Latenz und Verlust

metrics/
  logger.py          # schreibt events.csv und state.csv
  analysis.py        # berechnet die vier Metriken aus den Logs

experiments/
  sweep.py           # Monte-Carlo-Läufe, parallelisiert
  plots.py           # Abbildungen für den Bericht

tests/
  test_invariants.py
```

3.2 Die harte Regel

allocation/ darf **nichts** aus sim/, backends/ oder metrics/ importieren. Die Abhängigkeit läuft ausschließlich in eine Richtung. Prüfung: In einem leeren Verzeichnis, das nur allocation/ enthält, muss import allocation funktionieren.

Diese Regel ist der eigentliche Wert der ganzen Architektur. Sie garantiert, dass der Code, der in Webots und auf der Hardware läuft, bit-identisch derselbe ist wie der, mit dem die Parameter optimiert wurden. Ohne sie sind die Simulationsergebnisse nicht auf die Realität übertragbar, und die Kernaussage des Berichts bricht weg.

3.3 Die Backend-Schnittstelle

```

class RobotBackend(Protocol):
    def get_pose(self) -> tuple[float, float, float]: ...
    def get_battery(self) -> float:          # Ladezustand in [0, 1]
    ...
    def drive_to(self, x: float, y: float) -> None: ...
    def is_at_target(self) -> bool: ...
    def broadcast(self, msg: bytes) -> None: ...
    def receive(self) -> list[bytes]: ...   # nicht blockierend
    def detect_rois(self) -> list[Detection]: ...
    def now(self) -> float: ...             # Sekunden seit Laufbeginn

```

Acht Methoden. Mehr braucht es nicht. Jede zusätzliche Methode erhöht das Risiko, dass Simulation und Hardware auseinanderlaufen.

Die Abstraktionsebene ist bewusst **Fähigkeit**, nicht **Treiber**. Nicht “schreibe Register 0x1A”, sondern “fahre zu Punkt (x, y)”. Die echte Pi-Puck-Bibliothek wird in `backends/pipuck.py` hinter dieser Schnittstelle versteckt. Dadurch wird der Mock trivial, statt zu einem Nachbau von I2C-Kommunikation zu werden.

4. Die Kostenfunktion — korrigiert

Die Formel aus dem ursprünglichen Proposal enthält zwei Fehler, die vor der Implementierung behoben werden müssen.

4.1 Fehler 1: inkonsistente Vorzeichen

Ursprünglich: $b_i = \alpha \hat{d}_i + \beta \hat{w}_i - \gamma \hat{e}_i - \delta p$, wobei das höchste Gebot gewinnen soll. Durchgerechnet bedeutet das: wer weiter weg ist, gewinnt; wer schon ausgelastet ist, gewinnt; wer volle Batterie hat, verliert. Alle drei Effekte sind falsch herum.

Die Formel ist eine **Kostenfunktion**. Also wird die Konvention umgedreht: **das niedrigste Gebot gewinnt**. Das entspricht auch der Konvention in der SSI-Auktionsliteratur und lässt sich im Bericht sauber zitieren.

4.2 Fehler 2: δ ist nicht identifizierbar

p wird mit der RoI angekündigt und ist für alle Bieter derselben Auktion identisch. Der Term δp verschiebt daher **alle** Gebote um denselben Betrag. Der Gewinner ändert sich nicht, egal welchen Wert δ annimmt. Ein Parametersweep über δ würde reines Rauschen messen.

Damit Priorität wirkt, braucht es einen **auktionsübergreifenden** Vergleich, also Preemption.

4.3 Endgültige Formulierung

Gebotskosten:

$$c_i(\tau) = \alpha \cdot \hat{d}_i(\tau) + \beta \cdot \hat{w}_i + \gamma \cdot (1 - \hat{e}_i)$$

mit

- $\hat{d}_i(\tau) = \min\left(\frac{\|\mathbf{x}_i - \mathbf{x}_\tau\|}{d_{\max}}, 1\right)$, wobei $d_{\max}$ die Arenadiagonale ist
- $\hat{w}_i = \min\left(\frac{|\mathcal{T}_i^{\text{aktiv}}|}{w_{\text{cap}}}, 1\right)$ mit $w_{\text{cap}} = 3$
- $\hat{e}_i \in [0, 1]$ Ladezustand, 1 = voll. Der Term $(1 - \hat{e}_i)$ sorgt dafür, dass leere Akkus **hohe** Kosten erzeugen

Alle drei Komponenten liegen in $[0, 1]$, sind also vergleichbar skaliert.

Normierung: $\alpha \equiv 1$. Begründung: Innerhalb einer Auktion werden Gebote nur relativ verglichen, eine gemeinsame Skalierung ändert die Rangfolge nicht. Von vier Parametern bleiben damit zwei freie (β, γ) plus δ separat. Das ist der Unterschied zwischen einem Parametergitter, das in 25 Minuten durchläuft, und einem, das nicht durchläuft.

Gewinnerbestimmung: Gebote aufsteigend sortieren, die n_{req} günstigsten gewinnen. Bei Gleichstand entscheidet die kleinere Roboter-ID — deterministisch, damit Läufe reproduzierbar bleiben.

Preemption mit Hysterese: Ein Roboter mit laufender Aufgabe τ_{cur} wechselt zu τ_{neu} genau dann, wenn

$$\delta \cdot (p_{\text{neu}} - p_{\text{cur}}) - (c_i(\tau_{\text{neu}}) - c_i(\tau_{\text{cur}})) > \theta_{\text{hyst}}$$

Der Prioritätsgewinn, gewichtet mit δ , muss den Kostenzuwachs um mehr als die Hysterese übersteigen. Nun ist δ identifizierbar und ein sinnvoller Sweep-Parameter. Startwert: $\theta_{\text{hyst}} =$

0,1.

Beim Wechsel sendet der Roboter RELEASE für die alte Aufgabe, die dadurch neu ausgeschrieben wird.

4.4 Wichtig: p ist keine Konstante

In der ursprünglichen Fassung war p ein skalarer Wert, der einmal mit der RoI angekündigt wurde. Das gilt nicht mehr. Die Priorität ist eine **Funktion der Wartezeit** und wird von jedem Roboter lokal ausgewertet. Die Herleitung, die Parameter und die Folgen für das Protokoll stehen im nächsten Abschnitt, der vor der Implementierung von Arbeitspaket 1 gelesen werden muss.

5. Prioritätsdynamik

Zwei Erweiterungen gegenüber dem ursprünglichen Proposal, die sich in **eine** Funktionsfamilie fassen lassen:

1. Bereiche, die lange nicht beobachtet wurden, werden von selbst wieder zu Aufgaben.
2. Verschiedene Task-Typen haben unterschiedliche Grundwertigkeit.

5.1 Die gemeinsame Formel

$$p(\Delta t) = p_0 + (1 - p_0) \left(1 - e^{-(\lambda \Delta t)^k}\right)$$

Rolle	p_0	Δt	Bedeutung von λ
SURVEY-Zelle	0	Zeit seit letzter Beobachtung	erwartete Ereignisrate je Zelle
RoI-Task (Push/Surround/Guard)	$p_{\text{base}}(\text{Typ})$	Zeit seit Ankündigung	Aging-Rate gegen Aushungerung

Eigenschaften, die für die Architektur zwingend sind: $p \in [0, 1]$ ist garantiert (das Preemptionskriterium setzt das voraus), p wächst streng monoton, und p sättigt — eine seit zehn Minuten unbesuchte Ecke kann nicht dauerhaft alles andere verdrängen.

5.2 Warum diese Form und keine andere

Für $k = 1$ ist die Formel **hergeleitet**, **nicht gewählt**. Treten Ereignisse in einer Zelle als Poisson-Prozess mit Rate λ auf, so ist

$$P(\text{mindestens ein unentdecktes Ereignis nach } \Delta t) = 1 - e^{-\lambda \Delta t}$$

Die Priorität einer SURVEY-Zelle *ist* damit die Wahrscheinlichkeit, dass dort etwas Unentdecktes liegt. λ hat eine physikalische Bedeutung und ist kein Tuning-Knopf. Für den Bericht ist das ein deutlicher Unterschied: eine begründete Modellannahme statt einer Heuristik.

Der Exponent k modelliert das Gedächtnis des Prozesses:

- $k = 1$ — gedächtnislos. Ein Ereignis ist zu jedem Zeitpunkt gleich wahrscheinlich. Angemessen für zufällige Neufunde.
- $k > 1$ — Alterung (Weibull-Form). Die Ereignisrate selbst wächst mit der Zeit. Angemessen für fortschreitende Prozesse: Nachbeben destabilisieren die Struktur, ein Brand breitet sich aus.

k ist damit eine dritte Sweep-Dimension, die praktisch nichts kostet.

Auslegung von λ : Halbwertszeit $t_{1/2} = \ln 2 / \lambda$. Für einen Lauf der Dauer T sollte $t_{1/2} \approx T/3$ gelten. Bei $t_{1/2} \ll T$ sind sofort alle Zellen maximal dringlich und die Priorisierung verliert ihre Wirkung; bei $t_{1/2} \gg T$ wird nie eine Zelle dringlich.

5.3 Folge für das Protokoll — nicht optional

Da p sich während der Laufzeit ändert, kann ANNOUNCE keinen skalaren Prioritätswert mehr tragen. Periodisches Neuausschreiben scheidet aus, weil das Nachrichtenaufkommen explodieren würde und ausgerechnet der Kommunikations-Overhead eine der vier Metriken ist.

Lösung: ANNOUNCE trägt die **Parameter der Prioritätsfunktion**, jeder Empfänger wertet sie lokal zur Gebotszeit aus.

ANNOUNCE: task_id, pos, type, n_req, p0, lam, k, t_ref, deadline, sender, t

Kosten: vier zusätzliche Floats pro Ankündigung, null zusätzliche Nachrichten. Preemptionsprüfungen benutzen damit automatisch den aktuellen Wert.

Zusätzlich wird der zum Ankündigungszeitpunkt gültige Wert $p(0) = p_0$ ins Log geschrieben — nur für Auswertung und Fehlersuche, nicht für die Entscheidungslogik.

5.4 SURVEY als vierter Task-Typ

Veraltete Zellen werden als eigener Task-Typ SURVEY mit $n_{req} = 1$ modelliert, nicht als Sonderfall im Explorationsverhalten. Dadurch verschmelzen Exploration und Aufgabenzuweisung zu einem einzigen Mechanismus.

Zwei Fallstricke, die zwingend behandelt werden müssen:

(a) Der Lastterm verliert seine Wirkung. Wenn ständig SURVEY-Tasks offen sind, ist jeder Roboter dauerhaft beschäftigt, $\hat{w}$ sättigt bei 1 und β kann nicht mehr unterscheiden. Regel: **SURVEY-Tasks zählen nicht in $\hat{w}$ und sind jederzeit unterbrechbar.** Sie sind Hintergrundarbeit, kein Auftrag.

(b) Zellenzahl. Das Coverage-Raster mit 5 cm ergibt in einer 2×2 m großen Arena 1600 Zellen und damit 1600 potenzielle Aufgaben. Deshalb ein **zweites, gröberes SURVEY-Raster** mit etwa 40 cm Kantenlänge (rund 25 Zellen). Pro Zeitschritt werden höchstens die $N = 3$ dringlichsten Zellen angekündigt, mit Cooldown je Zelle.

5.5 Basis-Prioritäten je Task-Typ

Die Rangfolge braucht eine Begründung, keine Zahlenwahl. Zwei in sich stimmige Varianten:

Begründungslinie	Push	Surround	Guard
Aufwand und Blockadewirkung	0,8	0,5	0,3
Rettungsertrag (SAR-Sicht)	0,4	0,8	0,6

Die SAR-Sicht argumentiert so: Surround grenzt eine Gefahr ein, deren Schaden mit der Zeit **wächst**; Guard markiert und überwacht eine lokalisierte Fundstelle, bis menschliche Kräfte eintreffen — im realen Urban Search and Rescue eine der wertvollsten Roboterleistungen; Push schafft Zugang und ist wichtig, aber selten unmittelbar lebensentscheidend.

Empfehlung: SAR-Ordnung als Hauptkonfiguration, Aufwand-Ordnung als Sensitivitätslauf. In Tier 0 kostet das eine Konfigurationszeile und liefert einen Diskussionsabsatz.

SURVEY erhält $p_0 = 0$ und liegt damit grundsätzlich unter jedem frisch angekündigten RoI-Task — bis die Wartezeit das umkehrt. Genau das ist beabsichtigt.

5.6 Aging gegen Aushungerung

Eine feste Rangfolge allein würde dazu führen, dass Guard bei ausgelasteter Flotte nie an die Reihe kommt. Der Term $(1 - p_0)(1 - e^{-(\lambda \Delta t)^k})$ verhindert das: p_0 ist der Startwert, nicht der Endwert. Nach hinreichender Wartezeit überholt jede Aufgabe jede frisch angekündigte. Das Prinzip entspricht dem *aging* in Betriebssystem-Schedulern und ist als solches zitierbar.

5.7 Preemptionskaskaden

Mit dynamischen Prioritäten wird ein bisheriger Randfall zum Regelfall: Ein hochprioritärer Surround-Task mit $n_{req} = 3$ reißt drei Roboter aus laufenden Aufträgen, erzeugt drei RELEASE, damit drei Neuausschreibungen — die ihrerseits Preemptionen auslösen können.

Regel, die den Großteil davon abfängt:

Preemption ist ausschließlich aus dem Zustand NAVIGATE erlaubt, niemals aus WAIT_PEERS oder EXECUTE.

Begründung, die direkt in den Bericht kann: Wer WAIT_PEERS verlässt, verbrennt nicht nur die eigene Anfahrtszeit, sondern auch die von $n_{req} - 1$ bereits wartenden Robotern. Der Schaden ist vielfach. Einsatztaktisch entspricht das dem Grundsatz, einen bereits formierten Trupp nicht auseinanderzureißen.

Zusätzlich: Cooldown je Roboter von etwa 15 s zwischen zwei Preemptionen, und die Zahl der Preemptionen wird als Metrik mitgeführt. Wenn sie explodiert, ist es im Log sichtbar.

5.8 Ereignisinjektion — die interessanteste Messung

In Tier 0 sollen Ereignisse **tatsächlich** mit einer wahren Rate λ_{wahr} je Zelle erzeugt werden. Der im Bidding verwendete Modellparameter λ_{Modell} ist davon unabhängig einstellbar.

Damit lässt sich messen, wie robust das System gegenüber einer **falsch geschätzten Ereignisrate** ist. Das ist die realistischste Frage des ganzen Projekts, denn eine reale Einsatzleitung kennt λ nie. Untersuchte Fälle: $\lambda_{\text{Modell}} = \lambda_{\text{wahr}}$, Unterschätzung um Faktor 3, Überschätzung um Faktor 3.

5.9 Was das für den Bericht bedeutet

Das Thema verschiebt sich von reiner Aufgabenzuweisung zu **persistenter Überwachung mit marktbasierter Zuweisung**. Das ist die schärfere und besser abgrenzbare Fragestellung. Zusätzliche Literatur für die Related-Work-Sektion:

- Idleness-basiertes Multi-Agenten-Patrolling (Machado et al.)
- Persistent monitoring mit mobilen Agenten (Smith, Schwager, Rus)
- Aging in Scheduling-Verfahren als Standardlösung gegen Aushungerung

Ehrlichkeitsgebot für die Results-Sektion: Die Coverage-Rate ist damit keine Nebenwirkung mehr, sondern eine gesteuerte Zielgröße. Der Vergleich mit Greedy muss das offenlegen, da Greedy kein SURVEY-Konzept besitzt. Sauber ist es, Greedy in zwei Varianten zu führen — mit und ohne SURVEY-Tasks.

5.10 Einschränkung für das Labor

Für die Läufe mit echter Hardware wird die persistente Überwachung abgeschaltet oder λ sehr klein gesetzt. Bei einer Arena von 2×2 m und Laufzeiten um fünf Minuten ist keine Zelle je lange genug unbeobachtet, als dass das Verhalten aussagekräftig würde. Die knappe Laborzeit gehört dem Strategievergleich.

Formulierung für den Bericht: Die persistente Überwachung wird ausschließlich in Simulation untersucht, weil Arenagröße und Laufdauer der realen Plattform den relevanten Zeitbereich nicht abdecken.

6. Arbeitspaket 1 — allocation/

6.1 tasks.py

```
class TaskType(Enum):
    PUSH = "push"           # n_req = 2..3
    SURROUND = "surround"   # n_req = 3
    GUARD = "guard"         # n_req = 1
    SURVEY = "survey"       # n_req = 1, Hintergrundaufgabe

class TaskState(Enum):
    OPEN, BIDDING, ASSIGNED, ACTIVE, DONE, FAILED

@dataclass(frozen=True)
class PriorityFn:
    """ $p(dt) = p_0 + (1-p_0) * (1 - \exp(-(lam*dt)**k))$ , siehe Abschnitt 5."""
    p0: float
    lam: float
    k: float
    t_ref: float           # Bezugszeitpunkt: Ankündigung bzw. letzte Beobachtung

    def at(self, t: float) -> float: ...

@dataclass(frozen=True)
class Task:
    task_id: str
    task_type: TaskType
    position: tuple[float, float]
    n_req: int
    priority_fn: PriorityFn
    duration: float        # Ausführungsdauer nach Eintreffen aller Roboter
    announced_at: float

    @property
    def counts_toward_workload(self) -> bool:
        return self.task_type is not TaskType.SURVEY
```

duration modelliert, wie lange die eigentliche Tätigkeit am Ort dauert, nachdem alle benötigten Roboter eingetroffen sind. Bei GUARD ist das die Haltezeit, bei PUSH die Schiebedauer. Vorschlag: GUARD 15 s, SURROUND 10 s, PUSH 20 s, SURVEY 2 s (kurzer Blick).

priority_fn ersetzt den früheren skalaren Wert priority. Die Basiswerte p_0 je Task-Typ stehen in Abschnitt 5.5, SURVEY erhält $p_0 = 0$.

6.2 bidding.py

Eine reine Funktion, ohne Seiteneffekte:

```
def compute_cost(robot_pose, battery, workload, task,
                 weights: BidWeights, d_max: float) -> float
```

BidWeights ist ein frozen dataclass mit alpha=1.0, beta, gamma, delta, theta_hyst.

Dazu die Preemptionsprüfung als eigene Funktion. Beide Funktionen sind trivial testbar, weil sie zustandslos sind — das ist Absicht.

6.3 messages.py

Fünf Nachrichtentypen, JSON-serialisiert. Feste Felder, keine optionalen Erweiterungen:

Typ	Felder	Bedeutung
ANNOUNCE	task_id, pos, type, n_req, p0, lam, k, t_ref, deadline, sender, t	Auktion eröffnet
BID	task_id, cost, sender, t	Gebot
AWARD	task_id, winners[], sender, t	Zuschlag
DONE	task_id, sender, t	Aufgabe abgeschlossen
RELEASE	task_id, sender, reason, t	Aufgabe zurückgegeben

deadline ist der Zeitpunkt, bis zu dem Gebote angenommen werden. Startwert für das Gebotsfenster: $T_{bid} = 1,0$ s.

Die Felder p0, lam, k, t_ref beschreiben die Prioritätsfunktion (Abschnitt 5.3). Es wird **kein** skalarer Prioritätswert übertragen — jeder Empfänger wertet die Funktion lokal zum Gebotszeitpunkt aus. Das ist die einzige Möglichkeit, zeitabhängige Prioritäten ohne wiederholtes Ausschreiben abzubilden.

Serialisierung: kompakt (json.dumps mit separators=(", ", ": ")), damit die Nachrichtengröße realistisch bleibt. Die Metrik "Communication overhead" zählt sowohl Anzahl als auch Byte-Summe.

6.4 auction.py

Der endliche Automat pro Roboter. Zustände:

```
EXPLORE → (Detektion) → ANNOUNCE → COLLECT_BIDS → AWARD → EXPLORE
EXPLORE → (ANNOUNCE empfangen) → BID_SENT → (AWARD empfangen)
                                     ↓ gewonnen           ↓ verloren
NAVIGATE → WAIT_PEERS → EXECUTE → EXPLORE
```

Ein Roboter kann gleichzeitig Auktionator und Bieter sein — er darf auf die eigene Ankündigung bieten. Das ist gewollt: meist ist der Entdecker tatsächlich am nächsten.

WAIT_PEERS ist bei $n_{req} > 1$ nötig: Die Ausführung beginnt erst, wenn **alle** zugeteilten Roboter am Ziel sind. Das ist der Punkt, an dem sich Push und Surround von Guard unterscheiden, und der Grund, warum Lastverteilung überhaupt zählt.

Randfälle, die explizit behandelt werden müssen:

- Weniger Gebote als n_{req} → nach Ablauf des Fensters erneut ausschreiben, maximal $R = 3$ Versuche, danach FAILED
- AWARD geht verloren → Bieter fällt nach Timeout zurück auf EXPLORE
- Auktionator fällt aus (Batterie leer) → Aufgabe verfällt nach Timeout, wird von einem anderen Roboter neu ausgeschrieben, sobald sie erneut detektiert wird
- Doppelte Ankündigung derselben RoI durch zwei Roboter → Deduplizierung über räumliche Nähe (Schwelle 0,15 m) und kleinere task_id gewinnt
- Preemption nur aus NAVIGATE, nie aus WAIT_PEERS oder EXECUTE (Abschnitt 5.7)
- Cooldown von 15 s zwischen zwei Preemptionen desselben Roboters
- Ein Roboter in einem SURVEY-Task gilt als frei: er bietet weiter und gibt den SURVEY-Task ohne Hysterese ab, sobald er einen RoI-Task gewinnt

Diese Randfälle sind kein Beiwerk. Sie sind genau das, was in einer realen Operation mit unzuverlässigem Funk ständig passiert, und ihre Behandlung ist ein Absatz wert.

6.5 strategies.py

Drei Strategien hinter derselben Schnittstelle:

- MarketStrategy: vollständige Auktion wie oben
- GreedyStrategy: Gebot ist ausschließlich die Distanz — implementiert als BidWeights(1, 0, 0, 0) mit deaktivierter Preemption
- RandomStrategy: Gebot ist eine Zufallszahl aus der laufübergreifend geseedeten Quelle

Wichtig: Greedy und Random durchlaufen dasselbe Nachrichtenprotokoll. Sonst wäre der Vergleich der Metrik “Communication overhead” wertlos, weil der Unterschied nur die Protokollimplementierung abbilden würde statt die Strategie.

7. Arbeitspaket 2 — sim/ und backends/abstract.py

7.1 Bewegungsmodell

Kein Differentialantrieb. Ein Roboter, der zu einem Ziel fährt, braucht

$$t = a + \frac{d}{v_{\text{eff}}}$$

a und v_{eff} werden in Phase 2 gegen Webots kalibriert. Bis dahin Startwerte $a = 1,2$ s, $v_{\text{eff}} = 0,08$ m/s. Die Position wird linear entlang der Verbindungsgeraden interpoliert; Hindernisse und andere Roboter werden ignoriert.

Warum das vertretbar ist: In einer weitgehend offenen Arena macht Hindernisvermeidung wenige Prozent der Fahrzeit aus, während Zuweisungsentscheidungen die Fahrzeit um Faktoren verändern. Und für den Fall, dass diese Annahme nicht trägt, existiert Tier 1.

7.2 Batteriemodell

$$\dot{e} = -(k_{\text{idle}} + k_{\text{move}} \cdot v)$$

Zwei Konstanten, linear. Startwerte so wählen, dass ein Roboter bei Dauerfahrt in etwa 20 Minuten leer ist — das entspricht der Größenordnung der realen Plattform und macht den Batterieterm in Läufen dieser Länge überhaupt wirksam.

Ein Roboter mit $\hat{e} \leq 0,05$ wechselt in DEPLETED, bietet nicht mehr und gibt laufende Aufgaben per RELEASE zurück. Das ist die Modellierung des Szenarios, das der Batterieterm gerade verhindern soll.

7.3 Detektionsmodell

Ein Roboter detektiert eine RoI, wenn sie innerhalb des Radius $r_{\text{det}} = 0,3$ m liegt. Optional mit Falsch-Negativ-Rate p_{miss} als Sensitivitätsparameter — dieser Parameter erlaubt später die Aussage, wie robust die Strategien gegenüber unzuverlässiger Wahrnehmung sind.

Ein Blickrichtungskegel ist optional und sollte nur eingebaut werden, wenn im Labor tatsächlich mit Kameradetektion gearbeitet wird.

7.4 Kommunikationsmodell

sim/comms.py: Nachrichten werden mit Latenz $L \sim \mathcal{U}(l_{\text{min}}, l_{\text{max}})$ zugestellt und mit Wahrscheinlichkeit p_{loss} verworfen. Startwerte $l_{\text{min}} = 5$ ms, $l_{\text{max}} = 40$ ms, $p_{\text{loss}} = 0,02$. Diese Werte werden nach dem Pilot-Laborslot durch die dort gemessene Latenz ersetzt.

Optional, aber wertvoll für die Diskussion: eine Reichweitenbegrenzung r_{comm} . Damit lässt sich das Szenario "Funkabschattung im Trümmerfeld" untersuchen — die Frage, ob die Auktion bei zerfallender Konnektivität degradiert oder zusammenbricht, ist im SAR-Kontext die interessanteste überhaupt und ein starker Kandidat für den Abschnitt "future improvements".

7.5 Zwei Raster: Coverage und SURVEY

Coverage-Raster, 5 cm. Nur für die Metrik. Eine Zelle gilt als abgedeckt, sobald ein Roboter innerhalb von r_{det} vorbeigekommen ist.

SURVEY-Raster, ca. 40 cm. Rund 25 Zellen bei einer 2×2 m großen Arena. Je Zelle wird t_{last} geführt und daraus $p(\Delta t)$ nach Abschnitt 5.1 berechnet. Angekündigt werden pro Zeitschritt höchstens die $N = 3$ dringlichsten Zellen, die weder zugewiesen sind noch im

Cooldown stehen.

Ein separates, gröberes Raster ist zwingend: Mit dem 5-cm-Raster entstünden 1600 potenzielle Aufgaben.

7.6 Ereignisinjektion

Je SURVEY-Zelle werden Ereignisse als Poisson-Prozess mit der wahren Rate λ_{wahr} erzeugt. Ein injiziertes Ereignis wird zu einer RoI, sobald ein Roboter die Zelle innerhalb von r_{det} passiert.

Der im Bidding verwendete Wert λ_{Modell} ist davon **unabhängig** konfigurierbar. Diese Trennung ist der Kern des Robustheitsexperiments aus Abschnitt 5.8 und darf im Code nicht zusammenfallen.

Protokolliert wird je Ereignis: Entstehungszeit, Zelle, Entdeckungszeit. Daraus ergibt sich die Detektionslatenz.

7.7 Explorationsverhalten

Mit SURVEY-Tasks ist ein separates Explorationsverhalten weitgehend überflüssig — die Erkundung ergibt sich aus dem Zuweisungsmechanismus. Als Rückfallebene für Roboter ohne jede Zuweisung: Fahrt zur nächstgelegenen Zelle mit maximalem p .

Für die Vergleichsbedingungen ohne SURVEY (Greedy-Variante A, siehe Abschnitt 5.9) wird ein Boustrophedon-Muster mit robotereigenem Sektor verwendet.

7.8 Zeitschleife

Fester Zeitschritt $\Delta t = 0,1$ s. Pro Schritt: Ereignisse injizieren → Nachrichten zustellen → SURVEY-Prioritäten aktualisieren und ggf. ankündigen → jeder Roboter aktualisiert seinen Automaten → Bewegung integrieren → Batterie aktualisieren → Detektion prüfen → Zustand protokollieren (alle 0,5 s).

Determinismus ist Pflicht. Eine einzige `random.Random(seed)`-Instanz pro Lauf, explizit durchgereicht. Niemals das globale `random`-Modul. Gleicher Seed muss byte-identische Logs erzeugen — das ist einer der Invariantentests und die Voraussetzung für gepaarte Vergleiche.

8. Arbeitspaket 3 — metrics/

8.1 Zwei CSV-Dateien pro Lauf

events.csv mit den Spalten `t`, `event_type`, `task_id`, `robot_id`, `payload_json` und state.csv mit `t`, `robot_id`, `x`, `y`, `battery`, `workload`, `state`.

Dieses Format gilt **unverändert** für Tier 0, Webots und die echten Roboter. Alle Metriken werden ausschließlich daraus berechnet. Das bedeutet: ein einziges Auswertungsskript für drei Datenquellen — und es bedeutet, dass ein Vergleich zwischen den Ebenen überhaupt möglich ist.

8.2 Die vier Metriken

Metrik	Berechnung aus den Logs
Task-Bearbeitungszeit	$t(\text{DONE}) - t(\text{ANNOUNCE})$ je Aufgabe; Median und Mittelwert
Coverage-Rate	abgedeckte Rasterzellen pro Sekunde, aus state.csv
Lastungleichheit	Standardabweichung der Aufgabenzahl je Roboter am Laufende
Kommunikations-Overhead	Anzahl und Byte-Summe der Nachrichten, geteilt durch die Zahl abgeschlossener Aufgaben

8.3 Metriken der persistenten Überwachung

Durch die Prioritätsdynamik kommen vier Metriken hinzu, alle aus denselben Logs berechenbar:

Metrik	Berechnung	Wozu
Mittlere Zellen-Idleness	Mittel über alle Zellen und Zeitpunkte von $t - t_{\text{last}}$	Standardmaß der Patrolling-Literatur
Maximale Zellen-Idleness	Maximum desselben Ausdrucks über den Lauf	Deckt Aushungerung auf; das Mittel allein verbirgt sie
Detektionslatenz	Entdeckungszeit minus Entstehungszeit je injiziertem Ereignis	Die eigentliche SAR-Zielgröße
Preemptionszahl	Anzahl RELEASE mit Grund preempted	Frühwarnung für Kaskaden

Zusätzlich erfassen (kostet nichts, hilft der Diskussion): Anteil fehlgeschlagener Aufgaben, mittlerer Ladezustand am Laufende, Zeitpunkt des ersten leergefahrenen Roboters, maximale Wartezeit eines abgeschlossenen Guard-Tasks (prüft, ob das Aging wirkt).

9. Invarianten und Tests

Diese elf Tests sind nicht optional. Sie fangen genau die Fehlerklasse, die in generiertem Code sonst unbemerkt bleibt — Code, der plausibel aussieht und läuft, aber das Falsche berechnet. Zugleich sind sie ein guter halber Absatz in der Implementation-Sektion.

1. **Greedy-Äquivalenz.** Bei $\beta = \gamma = \delta = 0$ und deaktivierter Preemption muss die Marktstrategie **exakt** dieselben Zuweisungen liefern wie Greedy. Der wichtigste Test überhaupt: Er prüft, ob die Kostenfunktion richtig verdrahtet ist.
2. **Trivialfall.** Bei einem Roboter und einer Aufgabe liefern alle drei Strategien identische Ergebnisse.
3. **Erhaltung.** $|\text{angekündigt}| = |\text{abgeschlossen}| + |\text{fehlgeschlagen}| + |\text{offen am Ende}|$. Kein Task verschwindet, keiner wird doppelt gezählt.
4. **Kapazität.** Kein Roboter ist gleichzeitig in mehr als w_{cap} Aufgaben eingetragen.
5. n_{req} -**Einhaltung.** Jede abgeschlossene Aufgabe hatte über die gesamte Ausführung exakt n_{req} zugeteilte Roboter.
6. **Determinismus.** Gleicher Seed $\rightarrow$ identischer SHA-256-Hash von `events.csv`.
7. **Monotonie.** Zeitstempel steigen monoton, keine negativen Zeiten, keine negativen Kosten.
8. **Prioritätsschranken.** Für alle $\Delta t \geq 0$ und alle Parameterkombinationen gilt $p_0 \leq p(\Delta t) < 1$, und p ist monoton wachsend. Property-based prüfen, nicht an drei Stützstellen.
9. **Kein Aushungern.** In einem Lauf mit ausreichender Dauer wird jeder angekündigte Task irgendwann zugewiesen. Prüft, ob das Aging tatsächlich greift.
10. **Preemptionsregel.** Kein RELEASE mit Grund preempted geht von einem Roboter im Zustand WAIT_PEERS oder EXECUTE aus.
11. **Workload-Ausschluss.** SURVEY-Tasks erhöhen $\hat{w}$ nicht. Ohne diesen Test bemerkt niemand, dass β wirkungslos geworden ist.

10. Vorgaben für den Quellcode

Der Appendix wird bewertet: 5 Punkte für Präsentationsqualität, 5 weitere dafür, ob die Implementierung korrekt, gut strukturiert und effizient ist — mit dem ausdrücklichen Hinweis, zu prüfen, ob alle Zeilen notwendig sind.

Verbindlich:

- PEP 8, Type Hints durchgängig, aussagekräftige Namen
- Docstrings, die das *Warum* erklären; keine Kommentare, die die Codezeile paraphrasieren
- dataclasses und Enum statt Dictionaries mit Stringschlüsseln
- Standardbibliothek bevorzugen: itertools, statistics, collections, pathlib
- logging statt print in Bibliothekscode
- Ein einziger geseedeter Zufallsgenerator, explizit durchgereicht
- NumPy für die Auswertung, Matplotlib für Abbildungen. Sonst nichts Schweres.

Zu vermeiden:

- Defensives try/except, das Fehler verschluckt. In der Simulation soll ein Fehler laut scheitern.
- Konfigurationsoptionen, die nie benutzt werden
- Abstraktionsschichten, die genau eine Implementierung haben (außer der Backend-Schnittstelle, die drei hat)
- Hilfsfunktionen mit einem einzigen Aufrufer

Richtgrößen: allocation/ unter 400 Zeilen, sim/ plus backends/abstract.py unter 350 Zeilen. Wenn deutlich mehr entsteht, wurde zu viel gebaut — dann kürzen, nicht rechtfertigen.

11. Reihenfolge der Arbeitspakete

WP	Inhalt	Fertig, wenn
WP0	PriorityFn inklusive Tests 8 und 9	Kurvenfamilie reproduziert Abschnitt 5.1, Schranken halten
WP1	allocation/tasks.py, bidding.py, messages.py	Kostenfunktion getestet, Nachrichten serialisieren und deserialisieren verlustfrei
WP2	allocation/auction.py — FSM inklusive aller Randfälle und Preemptionsregel	Zwei Roboter führen im Unit-Test eine vollständige Auktion durch, Test 10 grün
WP3	sim/ und backends/abstract.py, beide Raster, Ereignisinjektion	Lauf mit 5 Robotern und 10 Aufgaben terminiert unter 2 s
WP4	allocation/strategies.py	Test 1 (Greedy-Äquivalenz) ist grün
WP5	metrics/logger.py, analysis.py	Alle vier Metriken werden aus den CSVs berechnet
WP6	tests/test_invariants.py	Alle sieben Tests grün
WP7	experiments/sweep.py, plots.py	8.100 Läufe unter 30 Minuten, drei Abbildungen erzeugt
WP8	Robustheitsexperiment $\lambda_{\text{Modell}} \neq \lambda_{\text{wahr}}$	Detektionslatenz über drei Fehlschätzungsstufen ausgewertet

WP0 steht bewusst vor WP1: Die Prioritätsfunktion ist die einzige Komponente, von der alles andere abhängt. Nach WP3 und nach WP6 jeweils den erzeugten Code vollständig durchlesen und kürzen. Diese beiden Durchgänge sind eingeplante Arbeitszeit, keine Kür.

12. Glossar

Begriff	Bedeutung
RoI	Region of Interest — detektiertes Objekt, das eine Aufgabe auslöst
n_{req}	Anzahl gleichzeitig benötigter Roboter für eine Aufgabe
SSI-Auktion	Sequential Single-Item auction; Aufgaben werden nacheinander einzeln versteigert
MRTA	Multi-Robot Task Allocation
Preemption	Abbruch einer laufenden Aufgabe zugunsten einer dringlicheren
Hysterese	Schwelle, die verhindert, dass bei marginalen Unterschieden ständig umdisponiert wird
Reality Gap	Diskrepanz zwischen Simulationsergebnis und Verhalten realer Hardware
Tier 0 / 1 / 2	abstrakte Simulation / Webots / echte Pi-Pucks
Pareto-Front	Menge der Konfigurationen, bei denen keine Metrik verbessert werden kann, ohne eine andere zu verschlechtern
Idleness	Zeit, die eine Zelle unbeobachtet geblieben ist; Standardmaß im Multi-Agenten-Patrolling
Aging	Anwachsen der Priorität mit der Wartezeit, um Aushungerung zu verhindern
Poisson-Prozess	Modell für zufällig und unabhängig auftretende Ereignisse mit konstanter Rate λ
Weibull-Form	Verallgemeinerung mit Formparameter k ; $k > 1$ bedeutet mit der Zeit steigende Ereignisrate
Aushungerung	Dauerhafte Nichtberücksichtigung niedrig priorisierter Aufgaben
SAR	Search and Rescue
USAR	Urban Search and Rescue — Rettung in eingestürzten baulichen Strukturen